SQL Server Backend Development

Transactions, Stored Procedures & Dapper

A practical tutorial for ASP.NET Core developers

Purpose: Learn how SQL Server transactions work, when stored procedures make sense, and how Dapper connects SQL to C# models.

Note: Examples use SQL Server syntax and modern C# / ASP.NET Core patterns. The guide is deliberately practical and assumes you already know basic C#.

Reference Guide • 2026

Table of Contents

- 1. C# vs SQL: where each kind of logic belongs
- 2. SQL fundamentals
- 3. Transactions and ACID
- 4. BEGIN, COMMIT and ROLLBACK
- 5. XACT_ABORT, TRY/CATCH, @@TRANCOUNT and THROW
- 6. Isolation levels and deadlocks
- 7. Stored procedures
- 8. Parameters and returned results
- 9. CRUD with stored procedures
- 10. Building a full website with SPs
- 11. Dapper fundamentals
- 12. Dapper + stored procedures
- 13. Dapper + transactions
- 14. Multiple result sets and output parameters
- 15. EF Core vs Dapper vs ADO.NET
- 16. Recommended ASP.NET architecture
- 17. Complete mini project
- 18. Performance and common mistakes
- 19. Learning roadmap and cheat sheet

1. C# vs SQL: Where Each Kind of Logic Belongs

A backend normally has two broad kinds of work: application/business logic and data work. C# is usually the natural home for business rules. SQL is usually the natural home for filtering, joining, aggregating, sorting and modifying data.

Business logic in C#

```
if (teacher.IsSuspended)
    return;

if (teacher.StudentCount < 50)
    return;

await notificationService.SendAsync(teacher);
```

Data-intensive work in SQL

```
SELECT t.Id, t.Name, COUNT(DISTINCT s.Id) AS StudentCount
FROM Teachers t
JOIN Courses c ON c.TeacherId = t.Id
JOIN Students s ON s.CourseId = c.Id
WHERE c.IsActive = 1
GROUP BY t.Id, t.Name
HAVING COUNT(DISTINCT s.Id) >= 50;
```

You could fetch the rows and calculate everything in C#, but moving large amounts of data from SQL Server to the application can be wasteful. SQL Server is optimized for set-based data operations.

Mental model

```
Controller
   ↓
Service / business logic
   ↓
EF Core / Dapper / ADO.NET
   ↓
SQL Server
   ↓
Tables + indexes + views + stored procedures
```

Note: A stored procedure is not automatically better than C#. It is a database-side program. Use it when having the operation in SQL Server gives you a real advantage.

2. SQL Fundamentals

SELECT / filtering / ordering

```
SELECT Id, Name
FROM Users
WHERE IsActive = 1
ORDER BY Name;
```

JOIN

```
SELECT u.Name, c.Title
FROM Users u
JOIN Courses c ON c.TeacherId = u.Id
WHERE u.IsActive = 1;
```

Aggregation

```
SELECT CourseId, COUNT(*) AS StudentCount
FROM CourseStudents
GROUP BY CourseId
HAVING COUNT(*) > 50;
```

CTE

```
WITH ActiveCourses AS
```

```
(
    SELECT *
    FROM Courses
    WHERE IsActive = 1
)
SELECT *
FROM ActiveCourses
WHERE TeacherId = @TeacherId;
```

Before going deep into SPs, be comfortable with SELECT, JOIN, GROUP BY, HAVING, subqueries, CTEs, INSERT, UPDATE, DELETE and parameters.

3. Transactions and ACID

A transaction groups database operations into one logical unit. A classic example is transferring money: subtract from one account and add to another. Both operations should succeed together.

```
BEGIN TRANSACTION;

UPDATE Accounts
SET Balance = Balance - 30
WHERE Id = 1;

UPDATE Accounts
SET Balance = Balance + 30
WHERE Id = 2;

COMMIT TRANSACTION;
```

ACID

• Atomicity: all-or-nothing behavior.

• Consistency: a successful transaction leaves the database satisfying its rules and constraints.

• Isolation: concurrent transactions are controlled so they do not create unsafe observations.

• Durability: committed changes are designed to survive failures.

Note: Transactions are useful for orders, inventory, payments, creating related records, and any operation where partial success would be a problem.

4. BEGIN, COMMIT and ROLLBACK

BEGIN TRANSACTION starts a transaction. COMMIT makes its changes permanent. ROLLBACK undoes its changes.

```
BEGIN TRANSACTION;

UPDATE Accounts
SET Balance = Balance - 30
WHERE Id = 1;

UPDATE Accounts
SET Balance = Balance + 30
WHERE Id = 2;

-- Everything succeeded:
COMMIT TRANSACTION;
```

Rollback

```
BEGIN TRANSACTION;

UPDATE Accounts
SET Balance = Balance - 30
WHERE Id = 1;

-- Validation fails:
ROLLBACK TRANSACTION;
```

A transaction is scoped to the current database connection/session. When using Dapper, commands that should participate in the transaction must use the same connection and transaction object.

5. XACT_ABORT, TRY/CATCH, @@TRANCOUNT and THROW

SET XACT_ABORT ON

XACT_ABORT changes transaction error behavior. When ON, a run-time error generally causes the current transaction to be terminated and rolled back. It is commonly used for transactional SQL and stored procedures.

```
SET XACT_ABORT ON;
```

TRY/CATCH

```
BEGIN TRY
    -- code that may fail
END TRY
BEGIN CATCH
    -- error handling
END CATCH;
```

Common robust pattern

```
SET XACT_ABORT ON;

BEGIN TRY
    BEGIN TRANSACTION;

    UPDATE Accounts
    SET Balance = Balance - 30
    WHERE Id = 1;

    UPDATE Accounts
    SET Balance = Balance + 30
    WHERE Id = 2;

    COMMIT TRANSACTION;
END TRY
BEGIN CATCH
    IF @@TRANCOUNT > 0
        ROLLBACK TRANSACTION;

    THROW;
END CATCH;
```

@@TRANCOUNT

@@TRANCOUNT reports the number of active transactions for the current connection. The defensive check avoids rolling back when no transaction is active.

THROW

THROW re-raises the error so the calling application can receive the database exception. This is especially important if ASP.NET needs to log, handle or translate the error.

> Note: XACT_ABORT does not make every possible SQL error identical. SQL Server has different error classes and transaction states, so production code should be tested with the actual failures it can encounter.

6. Isolation Levels and Deadlocks

Atomicity is only one transaction concern. Concurrent transactions also need rules about what one transaction can see while another is changing data.

- READ UNCOMMITTED: permits dirty reads.

• READ COMMITTED: common default behavior; prevents dirty reads.

• REPEATABLE READ: protects rows already read from changes until the transaction ends.

• SNAPSHOT: uses row versions for consistent reads.

• SERIALIZABLE: strongest traditional isolation; can reduce concurrency.

Deadlocks

```
Transaction A:
UPDATE Accounts WHERE Id = 1;
-- waits for Id = 2

Transaction B:
UPDATE Accounts WHERE Id = 2;
-- waits for Id = 1
```

SQL Server detects the cycle and chooses a transaction as the deadlock victim. Good practices include keeping transactions short, accessing shared resources in a consistent order, using suitable indexes, and retrying only errors that are safe to retry.

7. Stored Procedures

A stored procedure (SP) is a named program stored in SQL Server. It can accept parameters, query or modify data, use variables and control flow, handle errors, use transactions, and return result sets.

Create

```
CREATE PROCEDURE dbo.GetUserById
    @UserId INT
AS
BEGIN
    SET NOCOUNT ON;

    SELECT Id, Name, IsActive
    FROM dbo.Users
    WHERE Id = @UserId;
END;
```

Execute

```
EXEC dbo.GetUserById @UserId = 123;
```

Parameters

```
CREATE PROCEDURE dbo.GetUsersByStatus
    @IsActive BIT
AS
BEGIN
    SET NOCOUNT ON;

    SELECT Id, Name
    FROM dbo.Users
    WHERE IsActive = @IsActive;
END;
```

Why SET NOCOUNT ON?

It suppresses the row-count messages SQL Server normally sends after statements. This reduces unnecessary message traffic and is common in application-facing procedures.

> Note: Always parameterize user input. Never build SQL by concatenating untrusted values.

8. Parameters and Returned Results

Input parameters

```
CREATE PROCEDURE dbo.CreateUser
    @Name NVARCHAR(100),
    @IsActive BIT
AS
BEGIN
    SET NOCOUNT ON;

    INSERT INTO dbo.Users (Name, IsActive)
    VALUES (@Name, @IsActive);
END;
```

OUTPUT parameter

```
CREATE PROCEDURE dbo.CreateUser
    @Name NVARCHAR(100),
    @NewUserId INT OUTPUT
AS
BEGIN
    SET NOCOUNT ON;

    INSERT INTO dbo.Users (Name)
    VALUES (@Name);

    SET @NewUserId = CONVERT(INT, SCOPE_IDENTITY());
END;
```

Result set

```
SELECT Id, Name, IsActive
FROM dbo.Users
WHERE IsActive = 1;
```

Multiple result sets

```
SELECT Id, Name
FROM dbo.Users
WHERE IsActive = 1;

SELECT Id, Title
FROM dbo.Courses
WHERE IsActive = 1;
```

A procedure can return rows, one scalar value, output parameters, and a return status. Keep the contract clear so the C# caller knows exactly what to expect.

9. CRUD with Stored Procedures

Create

```
CREATE PROCEDURE dbo.User_Create
    @Name NVARCHAR(100),
    @IsActive BIT
AS
BEGIN
    SET NOCOUNT ON;

    INSERT INTO dbo.Users (Name, IsActive)
    VALUES (@Name, @IsActive);

    SELECT CONVERT(INT, SCOPE_IDENTITY()) AS Id;
END;
```

Read

```
CREATE PROCEDURE dbo.User_GetById
    @Id INT
AS
BEGIN
    SET NOCOUNT ON;

    SELECT Id, Name, IsActive
    FROM dbo.Users
    WHERE Id = @Id;
```

```
END;
```

Update / Delete

```
CREATE PROCEDURE dbo.User_Update
    @Id INT,
    @Name NVARCHAR(100),
    @IsActive BIT
AS
BEGIN
    SET NOCOUNT ON;

    UPDATE dbo.Users
    SET Name = @Name, IsActive = @IsActive
    WHERE Id = @Id;
END;

CREATE PROCEDURE dbo.User_Delete
    @Id INT
AS
BEGIN
    SET NOCOUNT ON;

    DELETE FROM dbo.Users
    WHERE Id = @Id;
END;
```

Note: CRUD procedures are not automatically superior to EF Core or Dapper SQL. Use them deliberately rather than turning every tiny query into an SP.

10. Can a Full Website Be Built with SPs?

Yes. A complete website can technically use stored procedures for nearly all database operations.

```
ASP.NET Core
    ↓
Service Layer
    ↓
Dapper
    ↓
Stored Procedures
    ↓
SQL Server
```

Why teams choose this

• Centralizes SQL operations in the database.

• Provides a stable database-side API for multiple applications.

• Keeps complex data operations close to the data.

• Can fit environments with strong database administration and permission requirements.

Costs

• Business logic can become split between C# and SQL.

• Large systems can accumulate huge numbers of procedures.

• Database code needs disciplined version control and deployment.

• Cross-layer refactoring can become harder.

A practical boundary is: C# owns application/business orchestration; SQL owns data-intensive operations. An SP can represent a meaningful database operation without becoming your entire business layer.

11. Dapper Fundamentals

Dapper is a lightweight .NET object mapper. You write the SQL and Dapper maps result columns to C# objects. Unlike EF Core, it does not provide a full change-tracking ORM.

Model

```
public class User
{
    public int Id { get; set; }
    public string Name { get; set; } = "";
    public bool IsActive { get; set; }
}
```

Query

```
const string sql = @"
SELECT Id, Name, IsActive
FROM Users
WHERE IsActive = @IsActive;";

var users = await connection.QueryAsync<User>(
    sql,
    new { IsActive = true });
```

Single row

```
var user = await connection.QuerySingleOrDefaultAsync<User>(
    @"
    SELECT Id, Name, IsActive
    FROM Users
    WHERE Id = @Id;",
    new { Id = userId });
```

Write

```
await connection.ExecuteAsync(
    @"
    UPDATE Users
    SET Name = @Name
    WHERE Id = @Id;",
    new { Id = userId, Name = "Hadi" });
```

Dapper is attractive when you want explicit SQL, predictable database interaction and simple object mapping without a large ORM abstraction.

12. Dapper + Stored Procedures

Dapper can call a stored procedure by setting CommandType.StoredProcedure.

```
var user = await connection.QuerySingleOrDefaultAsync<User>(
    "dbo.GetUserById",
    new { UserId = 123 },
    commandType: CommandType.StoredProcedure);
```

DynamicParameters

```
var parameters = new DynamicParameters();
parameters.Add("@UserId", userId, DbType.Int32);

var user = await connection.QuerySingleOrDefaultAsync<User>(
    "dbo.GetUserById",
    parameters,
    commandType: CommandType.StoredProcedure);
```

The repository/data-access layer can hide the stored procedure name and parameter details from the service layer.

13. Dapper + Transactions

Create one connection, begin one transaction, execute all related commands using that same transaction, then commit or roll back.

```
await using var connection = new SqlConnection(connectionString);
await connection.OpenAsync();

await using var transaction =
    await connection.BeginTransactionAsync();

try
{
    await connection.ExecuteAsync(
        "dbo.CreateOrder",
        new { CustomerId = customerId },
        transaction: transaction,
        commandType: CommandType.StoredProcedure);

    await connection.ExecuteAsync(
        "dbo.CreateOrderItem",
        new { OrderId = orderId, ProductId = productId },
        transaction: transaction,
        commandType: CommandType.StoredProcedure);

    await transaction.CommitAsync();
}
catch
{
    await transaction.RollbackAsync();
    throw;
}
```

Note: The transaction: transaction argument matters. Commands must participate in the same transaction if you expect them to roll back together.

14. Multiple Result Sets and Output Parameters

QueryMultiple

```
using var multi = await connection.QueryMultipleAsync(
    "dbo.GetDashboardData",
    new { TeacherId = teacherId },
    commandType: CommandType.StoredProcedure);

var teacher = await multi.ReadSingleAsync<Teacher>();
var courses = (await multi.ReadAsync<Course>()).ToList();
var exams = (await multi.ReadAsync<Exam>()).ToList();
```

Output parameter

```
var p = new DynamicParameters();
p.Add("@Name", "Hadi");
p.Add("@NewUserId",
    dbType: DbType.Int32,
    direction: ParameterDirection.Output);

await connection.ExecuteAsync(
    "dbo.CreateUser",
    p,
    commandType: CommandType.StoredProcedure);

int newUserId = p.Get<int>("@NewUserId");
```

Scalar

```
var count = await connection.ExecuteScalarAsync<int>(
    "dbo.GetActiveUserCount",
    commandType: CommandType.StoredProcedure);
```

15. EF Core vs Dapper vs ADO.NET

Feature	EF Core	Dapper	ADO.NET
SQL control	Medium	High	Very high
Object mapping	Strong	Strong/simple	Manual
Change tracking	Yes	No	No
Boilerplate	Low/medium	Low	High
LINQ	Yes	No	No
SP fit	Supported	Excellent	Excellent

Mental model: EF Core is a full ORM, Dapper is a lightweight SQL mapper, and ADO.NET is the lower-level database API. Dapper gives you substantial SQL control while still handling common mapping and parameter work.

16. Recommended ASP.NET Architecture

```
Controller
    ↓
Service / application layer
    ↓
Repository / data access
    ↓
Dapper
    ↓
Stored Procedure
    ↓
SQL Server
```

Controller

```
[HttpGet("{id:int}")]
public async Task<IActionResult> Get(int id)
{
    var user = await _userService.GetByIdAsync(id);

    if (user is null)
        return NotFound();

    return Ok(user);
}
```

Service

```
public Task<User?> GetByIdAsync(int id)
{
    return _userRepository.GetByIdAsync(id);
}
```

Repository

```
public Task<User?> GetByIdAsync(int id)
{
    return _connection.QuerySingleOrDefaultAsync<User>(
        "dbo.GetUserById",
        new { UserId = id },
        commandType: CommandType.StoredProcedure);
}
```

Note: Do not create abstractions merely to hide every line of SQL. The goal is a clean responsibility boundary, not maximum number of layers.

17. Complete Mini Project

Imagine an order system where creating an order and its items must be atomic.

Tables

```
CREATE TABLE dbo.Orders
(
    Id INT IDENTITY PRIMARY KEY,
    CustomerId INT NOT NULL,
    CreatedAtUtc DATETIME2 NOT NULL
);

CREATE TABLE dbo.OrderItems
(
    Id INT IDENTITY PRIMARY KEY,
    OrderId INT NOT NULL,
    ProductId INT NOT NULL,
    Quantity INT NOT NULL
);
```

One procedure that owns the whole operation

```
CREATE PROCEDURE dbo.Order_Create
    @CustomerId INT,
    @ProductId INT,
    @Quantity INT
AS
BEGIN
    SET NOCOUNT ON;
    SET XACT_ABORT ON;

    BEGIN TRY
        BEGIN TRANSACTION;

        INSERT INTO dbo.Orders (CustomerId, CreatedAtUtc)
        VALUES (@CustomerId, SYSUTCDATETIME());

        DECLARE @OrderId INT =
            CONVERT(INT, SCOPE_IDENTITY());

        INSERT INTO dbo.OrderItems
            (OrderId, ProductId, Quantity)
        VALUES
            (@OrderId, @ProductId, @Quantity);

        COMMIT TRANSACTION;

        SELECT @OrderId AS OrderId;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0
            ROLLBACK TRANSACTION;
        THROW;
    END CATCH;
END;
```

Dapper call

```
var orderId = await connection.ExecuteScalarAsync<int>(
    "dbo.Order_Create",
    new
    {
        CustomerId = customerId,
        ProductId = productId,
        Quantity = quantity
    },
    commandType: CommandType.StoredProcedure);
```

This design has a simple transaction boundary: the stored procedure owns creation of both the order and its first item. Another valid design is to let C# own a transaction spanning several procedure calls.

> Note: A transaction inside one stored procedure does not automatically include later, separate database calls from C#. Transaction scope must be deliberately shared.

18. Performance and Common Mistakes

Indexes

```
CREATE INDEX IX_Users_IsActive
ON dbo.Users(IsActive);
```

Indexes can make filtering, joining and sorting faster, but they consume storage and make writes more expensive. Index according to real query patterns.

Keep transactions short

Do not hold a transaction while waiting for an HTTP request, external API, user input or slow computation.

Never concatenate user input into SQL

```
-- Bad
var sql = "SELECT * FROM Users WHERE Name = '" + name + "'";

-- Good
var sql = "SELECT * FROM Users WHERE Name = @Name";
var users = await connection.QueryAsync<User>(
    sql, new { Name = name });
```

Avoid N+1 queries

Do not fetch 1,000 records and then perform 1,000 separate queries unless you have a very specific reason. Consider JOINs, batching or multiple result sets.

Do not make every query an SP

Stored procedures are a tool, not a requirement. Use them when they improve the design, security model, performance, reuse or operational requirements of your system.

19. Learning Roadmap and Cheat Sheet

Transaction

```
BEGIN TRANSACTION;
COMMIT TRANSACTION;
ROLLBACK TRANSACTION;

SET XACT_ABORT ON;

BEGIN TRY
    ...
END TRY
BEGIN CATCH
    IF @@TRANCOUNT > 0
        ROLLBACK TRANSACTION;
    THROW;
END CATCH;
```

Stored procedure

```
CREATE PROCEDURE dbo.Name
    @Parameter INT
AS
BEGIN
    SET NOCOUNT ON;

    SELECT ...
END;

EXEC dbo.Name @Parameter = 123;
```

Dapper

```
QueryAsync<T>                  -- many rows
QuerySingleAsync<T>            -- exactly one row
QuerySingleOrDefaultAsync<T>   -- zero or one row
ExecuteAsync                   -- write / command
```

```
ExecuteScalarAsync<T>          -- one value
QueryMultipleAsync             -- multiple result sets
```

Recommended learning order

- Master SELECT, JOIN, GROUP BY, HAVING, subqueries and CTEs.
- Learn indexes and basic execution-plan reading.
- Master transactions and isolation levels.
- Write stored procedures with parameters and error handling.
- Learn Dapper mapping and parameter binding.
- Combine Dapper + SPs + transactions.
- Build one real CRUD feature with the stack.
- Implement the same feature once with EF Core and once with Dapper/SPs; compare the trade-offs.

Final mental model

```
C#             = application behavior
SQL            = data operations
Stored Proc    = reusable database-side operation
Dapper         = lightweight SQL → C# mapper
Transaction    = atomic group of DB operations
XACT_ABORT     = stronger automatic transaction error behavior
TRY/CATCH      = SQL error-handling structure
COMMIT         = keep changes
ROLLBACK       = undo changes
```

Note: If you already know ASP.NET and EF Core, the fastest way to become comfortable with this stack is to build something real rather than memorizing syntax. Start with a small CRUD module, then add one multi-table transaction and one reporting-style query.